The program around the loop

Animated explainer · src/cli.js

agent.js is a library. Everything a person needs in order to run it lives here: argument parsing, the banner, the task prompt, the y/N confirmation, the demo and hook side doors, and the exit codes.

9 scenes · 4 exit codes · 3 side doors · MAX_HISTORY 24

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The surface area 4

03 · Argument parsing 5

04 · Two shapes of run 6

05 · Session memory 7

06 · Exit codes 8

07 · Human in the loop 9

08 · Side doors 10

09 · Recap 11

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

The loop is a library. This is the program.

SRC/CLI.JS · *source*

```
import { runTask } from './agent.js';

const result = await runTask({ ...base, task });
process.exitCode = taskExitCode(result);
```

What can I type? — Flags, --help, the task field.

What won? — The banner shows key, model and policy.

Side doors — --demo, --hooks, --memory.

> **remember** Nothing in agent.js knows a flag name, a colour, or an exit code. That separation is the design.
> **On screen:** agent.js answers: what is the loop? cli.js answers: what does a person type? It parses flags, loads config, builds the workspace, sandbox and memory, then calls runTask. The loop stays a library. The CLI owns every process concern around it.

02 · The surface area

The whole command surface.

`--demo` · `--yes / -y` · `--model` · `--max-steps` · `--allow-commands` · `--hooks` · `--memory` · `--no-memory` · `--trace-hooks` · `--no-hooks` · `--help / -h`

Run — A task argument runs once. No task means a session.

Bounds — --max-steps, --allow-commands, --yes.

Explain — --hooks prints the lifecycle; --memory prints the store.

Offline — --demo runs the whole loop with a scripted model.

> **On screen:** Every flag is parsed by hand. There is no command framework hiding the surface. The first group runs tasks and bounds them. The second group explains the program itself. Anything that is not a flag becomes the task, so quoting is optional.

03 · Argument parsing

Flags in. Task out.

SRC/CLI.JS · *source*

```
for (let i = 0; i < argv.length; i += 1) {
  const a = argv[i];
  if (a === '--demo') opts.demo = true;
  else if (a === '--model') opts.model = argv[++i];
  else if (a === '--max-steps') opts.maxSteps = Number(argv[++i]);
  else if (a === '--hooks') opts.showHooks = true;
  else if (a === '--memory') opts.showMemory = true;
  else if (a === '--help' || a === '-h') opts.help = true;
  else words.push(a);
}
opts.task = words.join(' ').trim();
```

No option library, no hidden defaults. The same loop also feeds the banner and the usage text.
On screen: parseArgs walks argv once. A flag sets an option; anything else is collected as the task. Flags that take a value consume the next argument. Everything else joins into the task words. The result is one plain object. That object is the whole command line surface.

04 · Two shapes of run

One shot, or stay in the room.

1. **Task?** — argument or --demo

one runTask

2. **Exit** — taskExitCode
3. **No task** — stdin is a TTY
4. **runSession** — task after task
5. **Carry history** — trimHistory

> No task and no TTY is not a crash: it is exit code 2, with a sentence explaining what to do instead.
> **On screen:** A task argument means one shot: run it, map the result to an exit code, leave. No task and a terminal means a session: read a task, run it, keep the conversation, ask again. No task and no terminal is exit code 2. The program says what to type instead of hanging.

05 · Session memory

Carry the conversation, not the system prompt.

SRC/CLI.JS · *source*

```
export function trimHistory(messages, max = MAX_HISTORY) {
  const kept = (messages ?? []).filter((m) => m.role !== 'system');
  let start = Math.max(0, kept.length - Math.max(0, max));
  while (start < kept.length && kept[start].role !== 'user') start += 1;
  return start >= kept.length ? [] : kept.slice(start);
}
```

- 🚫 Drop system it is rebuilt inside runTask.
- ✂️ Keep the tail at most MAX_HISTORY messages.
- 👤 Cut on user never start on a tool result or unanswered tool call.

> **On screen:** The next task needs the conversation, but not the old system prompt. trimHistory keeps the tail and starts on a user message. Providers reject any other boundary. That is why a follow-up like "now add tests" still knows what was just built.

06 · Exit codes

Failure gets a sentence.

0 — Finished. The model gave a final answer, or the session ended.

1 — The run did not finish: request failed, timeout, bad key, fatal error.

2 — No task, and stdin is not a terminal. Nothing ran.

3 — The step limit was reached before an answer.

> A non-zero exit is the last thing anyone reads, so it gets a plain-English note instead of a number.
> **On screen:** The CLI has four outcomes. Each one has a number and a reason. Exit 1 is a failed request, 2 is no way to ask, 3 is the step budget. main() prints the meaning of the code after the run, so the last line is actionable.

07 · Human in the loop

An unanswerable prompt is not approval.

SRC/CLI.JS · *source*

```
function makeConfirm() {
  return async (command) => {
    if (!process.stdin.isTTY) return 'deny';

    const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
    const answer = await new Promise((resolve) =>
      rl.question('run? ' + command + ' [y/N] ', resolve));
    rl.close();
    return /^y(es)?$/i.test(answer.trim()) ? 'allow' : 'deny';
  };
}
```

> Piped input degrades to deny. The ask policy asks a person, not a stream of bytes.
> **On screen:** When CMD_POLICY is ask, every command stops at a y/N prompt. No TTY means nobody can answer, so the answer is deny, never allow. Only an explicit yes runs the command. Everything else refuses.

08 · Side doors

Three doors that need no key.

--demo — A scripted model runs the real loop, tools and sandbox guards.

--hooks — Prints every lifecycle moment, icon, colour and reason.

--memory — Prints the file and exactly what the prompt injects.

NODE SRC/CLI.JS --DEMO · *text*

```
task      : Show the loop, then try to break the sandbox...
🚀 [HOOK:SessionStart] once, before the first model call
🧠 [HOOK:PreModel] before every model call - step 1/13
...
```

> The side doors describe the program without requiring a model, a key, or a network.
> **On screen:** Three flags exist only to explain the program: demo, hooks, and memory. --demo runs the real loop with a fake model. --hooks and --memory print their own lessons. None of them needs a key or a network, which makes them safe on stage.

09 · Recap

The CLI owns the edges.

• 4 exit codes

• 3 side doors

• 24 history max

→

Parse by hand. One pass over argv, no hidden command framework.

→

Map results to codes. 0, 1, 2 and 3 each have a sentence.

→

Ask a person. No TTY means deny, never allow.

→

Keep the conversation. trimHistory cuts only on a user message.

> **remember** The loop stays readable because this file owns everything around it.
> **On screen:** Four exit codes, three side doors, and one conversation carried between tasks. The CLI parses, explains, asks, bounds, and maps results to process codes. That is the whole program around the loop.